LAB05 – CONVOLUTION, PADDING VÀ CÁC BỘ LỌC CƠ BẢN

1. Mục tiêu

- Hiểu nguyên lý của phép Convolution 2D và vai trò của kernel trong xử lý ảnh.
- Phân biệt Convolution và Correlation; biết cách sử dụng cv2.filter2D().
- Hiểu và minh họa các phương pháp Padding: Constant/Zero, Replicate và Reflect.
- Áp dụng các bộ lọc Mean, Gaussian, Median, Sharpening và Emboss trên ảnh thực tế.
- So sánh ảnh hưởng của kích thước kernel và tham số bộ lọc đến chất lượng ảnh.
- Biết lựa chọn bộ lọc phù hợp với từng loại nhiễu và yêu cầu xử lý.

2. Yêu cầu chung

- Sử dụng Python với OpenCV (cv2), NumPy và Matplotlib.
- Mỗi bài phải hiển thị ảnh đầu vào và kết quả để so sánh.
- Các bài yêu cầu tự cài đặt không được thay thế bằng hàm OpenCV tương ứng.
- Sinh viên phải viết nhận xét ngắn sau mỗi bài: hiện tượng quan sát được, nguyên nhân và kết luận.
- Khi xử lý các kernel có thể sinh giá trị âm, nên sử dụng kiểu dữ liệu float32/CV_32F trước khi chuẩn hóa để hiển thị.

3. Hai loại nhiễu sử dụng trong LAB

Gaussian Noise: nhiễu cộng có giá trị ngẫu nhiên thường được mô hình hóa theo phân phối chuẩn. Tham số sigma càng lớn thì mức dao động cường độ càng mạnh.

Salt & Pepper Noise: một tỷ lệ pixel bị thay trực tiếp bởi các mức cực trị, thường là 0 (pepper) và 255 (salt). Tham số amount biểu diễn tỷ lệ pixel bị nhiễu.

Mục đích của việc tự tạo nhiễu là kiểm soát mức nhiễu và tạo điều kiện so sánh công bằng giữa Mean, Gaussian và Median Filter.

4. Bài tập thực hành

Thứ tự thực hiện theo tiến trình: Convolution → Padding → tạo dữ liệu nhiễu → các bộ lọc làm mượt/khử nhiễu → so sánh → làm sắc nét/hiệu ứng → pipeline tổng hợp → challenge.

Bài 1 – Convolution trên array

Cho ma trận ảnh A kích thước 5×5 và một kernel 3×3. Thực hiện convolution bằng hai cách để kiểm chứng kết quả.

1. Xây dựng hàm convolution2d(image, kernel): zero padding, lật kernel 180°, sliding window, nhân từng phần tử và tính tổng.
2. Thực hiện cùng phép toán bằng cv2.filter2D(). Vì filter2D thực hiện correlation, cần lật kernel trước khi truyền vào hàm nếu muốn đúng định nghĩa convolution.
3. In ma trận đầu vào, kernel, kernel sau khi lật và hai ma trận kết quả.

4. Dùng np.allclose() để kiểm tra hai kết quả có giống nhau hay không.
5. Chọn một vị trí output và trình bày chi tiết region × kernel rồi tính tổng.

```
def convolution2d(image, kernel):
    # TODO 1: lấy kích thước kernel
    # TODO 2: padding
    # TODO 3: lật kernel 180 độ
    # TODO 4: tạo output float32
    # TODO 5: sliding window
    # TODO 6: nhân và tính tổng
    pass
```

Yêu cầu nhận xét: Giải thích vì sao phải lật kernel trong convolution và sự khác nhau với correlation.

Bài 2 – Minh họa các phương pháp Padding

Cho array 3×3, thực hiện padding P=1 và quan sát giá trị được tạo quanh biên.

1. Thực hiện Zero/Constant Padding.
2. Thực hiện Replicate Padding.
3. Thực hiện Reflect Padding.
4. In các ma trận sau padding và hiển thị trực quan giá trị từng phần tử.
5. Dùng cùng Mean kernel 3×3 để lọc array theo ba phương pháp padding.
6. So sánh output tại pixel góc, pixel biên và pixel ở tâm.

Yêu cầu nhận xét: Padding ảnh hưởng mạnh nhất ở đâu? Constant padding có thể tạo hiện tượng gì ở biên ảnh?

Bài 3 – Tạo nhiễu cho ảnh

Trước khi khảo sát các bộ lọc khử nhiễu, sinh viên chủ động tạo ảnh nhiễu từ một ảnh sạch để có cùng dữ liệu đầu vào khi so sánh. Thực hiện tối thiểu Gaussian Noise và Salt & Pepper Noise.

1. Giữ lại các ảnh nhiễu để sử dụng thống nhất trong Bài 5, Bài 6 và Bài 7.
2. Hiển thị Original | Gaussian Noise | Salt & Pepper Noise trên cùng figure và ghi rõ tham số.
3. Tạo ít nhất 3 mức Gaussian Noise bằng cách thay đổi sigma và 3 mức Salt & Pepper Noise bằng cách thay đổi amount.
4. Viết hàm add_salt_pepper_noise(image, amount, salt_ratio) để tạo các điểm trắng (255) và đen (0) ngẫu nhiên.
5. Viết hàm add_gaussian_noise(image, mean, sigma) để cộng nhiễu Gaussian vào ảnh. Kết quả phải được clip về [0,255] và chuyển về uint8.
6. Đọc ảnh grayscale và chuyển về kiểu dữ liệu phù hợp trước khi tính toán.

Code:

```
def add_gaussian_noise(image, mean=0, sigma=20):
    # 1. Sinh noise theo phân phối Gaussian
    noise = np.random.normal(
        mean,
        sigma,
        image.shape
    )

    # 2. Chuyển ảnh sang float32 và cộng noise
```

```
    noisy = image.astype(np.float32) + noise

    # 3. Giới hạn giá trị về [0, 255]
    noisy = np.clip(noisy, 0, 255)

    # 4. Chuyển về uint8
    return noisy.astype(np.uint8)

def add_salt_pepper_noise(image, amount=0.02, salt_ratio=0.5):
    # 1. Sao chép ảnh
    noisy = image.copy()

    # 2. Tổng số pixel cần tạo nhiễu
    num_noise = int(amount * image.size)

    # Số pixel salt và pepper
    num_salt = int(num_noise * salt_ratio)
    num_pepper = num_noise - num_salt

    # 3. Chọn ngẫu nhiên vị trí Salt
    rows = np.random.randint(0, image.shape[0], num_salt)
    cols = np.random.randint(0, image.shape[1], num_salt)
    noisy[rows, cols] = 255

    # 4. Chọn ngẫu nhiên vị trí Pepper
    rows = np.random.randint(0, image.shape[0], num_pepper)
    cols = np.random.randint(0, image.shape[1], num_pepper)
    noisy[rows, cols] = 0

    # 5. Trả về ảnh
    return noisy
```

NHẬN XÉT BẮT BUỘC: Gaussian Noise và Salt & Pepper Noise khác nhau như thế nào về phân bố pixel? Khi tăng sigma hoặc amount, histogram và chất lượng ảnh thay đổi ra sao?

Bài 4 – Mean Filter

Đọc một ảnh thực tế và áp dụng bộ lọc trung bình với nhiều kích thước kernel.

1. Chuyển ảnh sang grayscale.
2. Tạo Mean kernel và thực hiện với k = 3, 5, 7.
3. Hiển thị Original | Mean 3×3 | Mean 5×5 | Mean 7×7 trên cùng một figure.
4. So sánh mức độ làm mượt, chi tiết và độ sắc nét khi tăng kích thước kernel.

```
def mean_filter(image, ksize):
    # TODO: tạo kernel có tổng hệ số bằng 1
    # TODO: áp dụng filter2D
    pass
```

Yêu cầu nhận xét: Kernel càng lớn thì ảnh thay đổi như thế nào? Vì sao?

Bài 5 – Gaussian Filter

Khảo sát ảnh hưởng của kích thước kernel và độ lệch chuẩn σ trong Gaussian smoothing.

1. Áp dụng Gaussian Filter với k = 3, 5, 7.
2. Cố định k = 7 và thử ít nhất ba giá trị σ khác nhau.
3. Hiển thị kết quả để so sánh.
4. So sánh Gaussian Filter với Mean Filter trên cùng ảnh.

Yêu cầu nhận xét: Vì sao Gaussian ưu tiên pixel gần tâm kernel? σ lớn hơn ảnh hưởng thế nào đến mức làm mượt?

Bài 6 – Median Filter và Salt & Pepper Noise

Khảo sát khả năng loại nhiễu xung của Median Filter.

1. Sử dụng ảnh Salt & Pepper Noise đã tạo ở Bài 3; thử tối thiểu hai mức amount khác nhau.
2. Áp dụng Median Filter với k = 3, 5, 7.
3. Trên cùng ảnh nhiễu, áp dụng thêm Mean và Gaussian để đối chiếu.
4. Hiển thị ảnh sạch, ảnh nhiễu và các ảnh sau lọc.
5. Chọn phương pháp phù hợp nhất với Salt & Pepper Noise và giải thích.

Yêu cầu nhận xét: Median khác Mean/Gaussian ở bản chất phép tính nào và vì sao điều này có lợi với nhiễu xung?

Bài 7 – So sánh Mean, Gaussian và Median

Tạo hai trường hợp nhiễu và đánh giá ba bộ lọc làm mượt trên cùng điều kiện.

1. Sử dụng Gaussian Noise và Salt & Pepper Noise đã tạo từ cùng ảnh gốc ở Bài 3.
2. Áp dụng Mean, Gaussian và Median với kích thước kernel tương đương.
3. Trình bày kết quả theo dạng bảng/figure để dễ so sánh.
4. Đánh giá khả năng khử nhiễu và khả năng giữ biên/chi tiết.
5. Lập bảng kết luận bộ lọc phù hợp hơn cho từng loại nhiễu.

Yêu cầu nhận xét: Không chỉ chọn ảnh 'đẹp nhất'; phải giải thích dựa trên đặc tính của từng bộ lọc.

Bài 8 – Sharpening

Thực hiện làm sắc nét ảnh bằng convolution với kernel sharpening.

1. Áp dụng kernel [[0,-1,0],[-1,5,-1],[0,-1,0]] bằng cv2.filter2D().
2. Hiển thị ảnh trước và sau sharpening.
3. Thử sharpening trên ảnh sạch và một ảnh có nhiễu.
4. So sánh vùng cạnh, chi tiết nhỏ và nhiễu sau xử lý.

Yêu cầu nhận xét: Sharpening có thể làm nhiễu mạnh hơn không? Giải thích.

Bài 9 – Emboss Filter

Áp dụng bộ lọc dập nổi và quan sát vai trò của giá trị âm/dương sau convolution.

1. Áp dụng kernel Emboss: [[-2,-1,0],[-1,1,1],[0,1,2]].
2. Sử dụng CV_32F để bảo toàn các giá trị âm và giá trị vượt 255.
3. Cộng 128 vào kết quả, clip về [0,255] và chuyển về uint8 để hiển thị.
4. Lật hướng kernel và quan sát sự thay đổi cảm giác nổi/lõm.
5. So sánh kết quả correlation và convolution với kernel Emboss.

```
result = cv2.filter2D(
    gray.astype(np.float32),
    cv2.CV_32F,
```

```
    kernel
)

# TODO: dịch mức xám
# TODO: clip [0,255]
# TODO: chuyển uint8
```

Yêu cầu nhận xét: Vì sao thường cộng 128 trước khi hiển thị ảnh Emboss?

Bài 10 – Pipeline khử nhiễu và làm sắc nét

Xây dựng một pipeline xử lý ảnh gồm bước denoise và bước sharpening.

1. Hoàn thiện hàm enhance_image().
2. Cho phép lựa chọn mean, gaussian hoặc median.
3. Thử ít nhất ba cấu hình bộ lọc/tham số.
4. Hiển thị ảnh input, sau denoise và sau sharpen.
5. Giải thích thứ tự xử lý được lựa chọn.

```
def enhance_image(image, denoise_type, ksize, sharpen=True):
    # TODO 1: chọn phương pháp denoise
    # TODO 2: thực hiện denoise
    # TODO 3: sharpen nếu được yêu cầu
    # TODO 4: xử lý miền giá trị
    # TODO 5: return
    pass
```

Yêu cầu nhận xét: Vì sao trong nhiều trường hợp nên khử nhiễu trước khi sharpening?

Bài 11 – Challenge: lựa chọn bộ lọc tự động

Xây dựng chương trình đơn giản tự lựa chọn bộ lọc cho ảnh đầu vào. Không yêu cầu sử dụng Machine Learning.

1. Chuẩn bị tập ảnh gồm ảnh sạch, Gaussian Noise và Salt & Pepper Noise.
2. Đề xuất ít nhất một đặc trưng/thống kê dùng để nhận biết tình trạng ảnh.
3. Xây dựng luật lựa chọn Mean/Gaussian/Median dựa trên đặc trưng đã chọn.
4. Thực hiện lọc và hiển thị kết quả.
5. Phân tích ít nhất một trường hợp chương trình lựa chọn sai hoặc chưa tối ưu.

Yêu cầu nhận xét: Đánh giá giới hạn của phương pháp luật thủ công và đề xuất hướng cải tiến.

5. Câu hỏi

1. Convolution khác Correlation ở bước nào?
2. Tại sao kernel Mean thường có tổng hệ số bằng 1?
3. Padding có làm thay đổi kết quả tại vùng giữa ảnh không? Trong trường hợp nào?
4. Tại sao Median Filter phù hợp với Salt & Pepper Noise?
5. Tại sao các kernel Sharpen/Emboss nên xử lý trên kiểu dữ liệu có dấu hoặc float?
6. Điều gì xảy ra nếu tăng kích thước kernel làm mượt quá lớn?
7. Vì sao sharpening sau denoising thường hợp lý hơn sharpening trước denoising?